

King Saud University – CCIS – CSC113

Project Title: Java project phase 1

Instructor: Dr Mohammed Faroun

Team Members:

1) Student Name (ID):

FARIS FALEH ALHARBI (445102358)

2) Student Name (ID):

FAISAL SAUD ALSAHLI (445100823)

3) Student Name (ID):

ABDULLAH OMAR ALMOKAITEEB (445100329)

Work Division:

1) FARIS: Worked on the implementation of the **RestaurantApp** class, the **Restaurant** class, and the design of the **UML class diagram**.

2) FAISAL: Worked on the implementation of the **Person**, **Employee**, **Waiter**, and **Customer** classes.

3) ABDULLAH: Worked on the implementation of the **Dish** and **Order** classes, and assisted with code testing and debugging.

Introduction

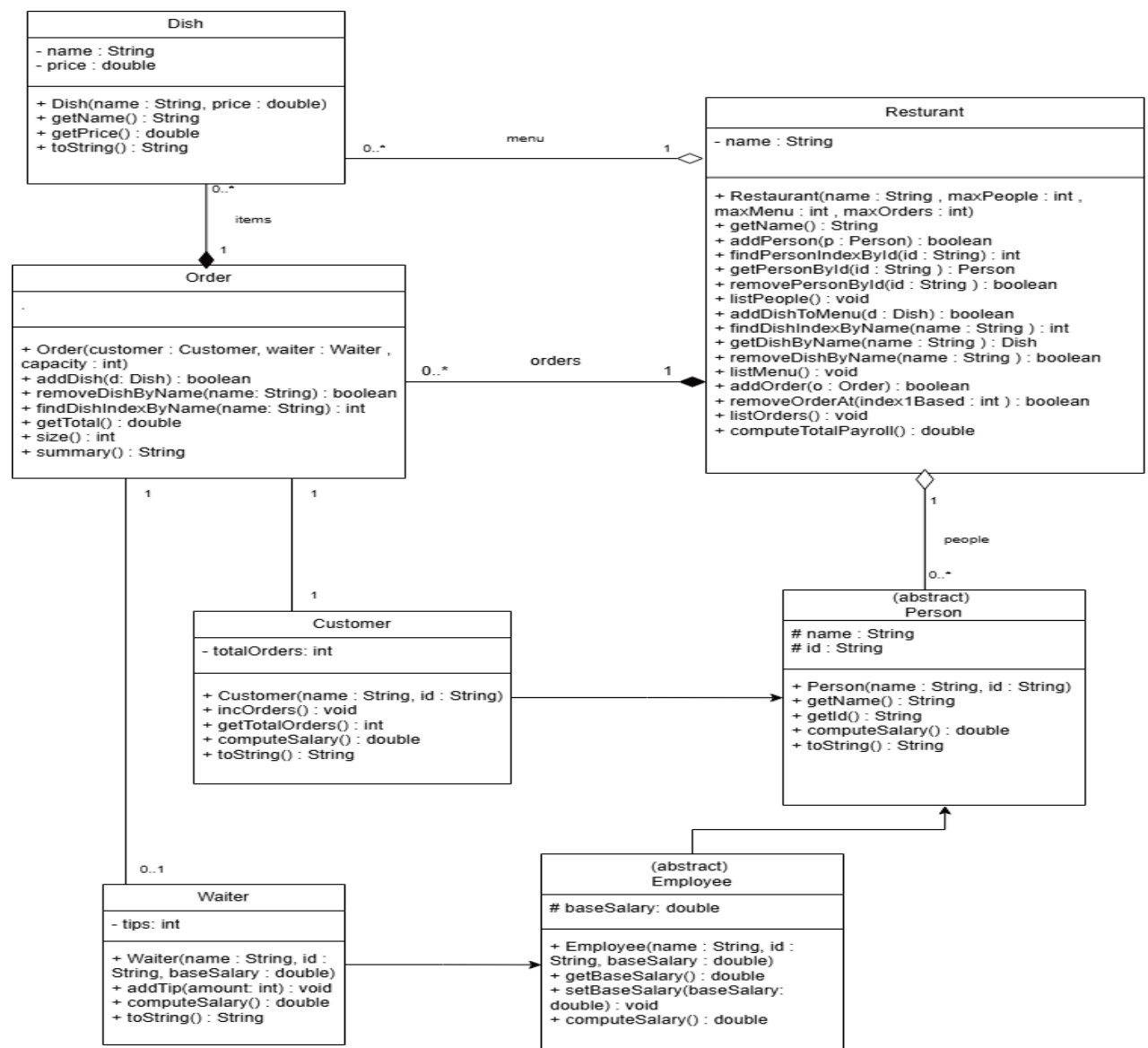
This project aims to apply the fundamental concepts of Object-Oriented Programming (OOP) covered in the CSC113 course.

The system represents a simple **Restaurant Management System** that allows managing customers, employees, dishes, and orders through an interactive console-based menu.

The main OOP principles demonstrated in this project are **inheritance**, **polymorphism**, **abstract classes**, **aggregation**, **composition**, and the use of **arrays of objects**.

All features are implemented in Java, following proper design and coding standards to ensure clarity, reusability, and extensibility of the system.

UML:



Implementation & Design Details

The project was implemented in Java using several classes that represent the main parts of a restaurant system.

Person is an abstract base class for all people, while Employee (also abstract) extends it and is inherited by Waiter.

Customer extends Person and keeps track of total orders.

Dish represents menu items with a name and price, and Order manages multiple dishes for a customer.

Restaurant handles all people, dishes, and orders, showing aggregation with Person and Dish, and composition with Order.

The RestaurantApp class contains the main method and provides a menu interface to test all program features.

The system applies inheritance, polymorphism, and OOP relationships effectively to create a clear and reusable design.

Sample Run:

```
=== KSU Restaurant Menu ===
1) Add Customer
2) Add Waiter
3) Add Dish to Menu
4) List Menu
5) Create Order
6) List Orders
7) List People
8) Compute Payroll
9) Remove Person by ID
10) Remove Dish by name
11) Remove Order by #
0) Exit
Choose:
```